Adaptive RDMA Optimization for Distributed LLM Workloads

Week 4: FlashAttention 3 Fix for NVIDIA Blackwell (RTX 5090)

Nathan Gopee
MS Computer Science, SUNY New Paltz
gopeen1@newpaltz.edu

Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

February 2026

1 Executive Summary

This update documents the discovery and resolution of a critical bug affecting FlashAttention 3 on NVIDIA Blackwell GPUs (RTX 5090). The fix enables our Cerberus node (2× RTX 5090, 64GB VRAM) to participate in multi-node vLLM inference with full FlashAttention 3 optimization.

Key Finding: A lazy import pattern resolves early CUDA initialization failures on Blackwell architecture, restoring ∼15% performance loss from FA2 fallback.

2 Problem Discovery

2.1 Initial Symptom

When attempting to run vLLM with FlashAttention 3 on Cerberus (RTX 5090), the following error occurred:

```
$ VLLM_FLASH_ATTN_VERSION=3 vllm serve deepseek-ai/DeepSeek-R1-Distill-Qwen-32B

AssertionError: Sinks are only supported in FlashAttention 3
RuntimeError: Early CUDA initialization error
```

This was unexpected because:

- RTX 5090 has compute capability 10.0+ (Blackwell)
- FlashAttention 3 documentation states support for compute capability ≥ 8.0
- The same configuration works on RTX 3090 (Ampere, compute capability 8.6)

2.2 Research Path

The fix was discovered through systematic investigation:

1. **vLLM gpt-oss Blog Post** — Confirmed gpt-oss requires FlashAttention with MXFP4
2. **Kernel Fusion Article** — Understood FlashInfer's role in MoE optimization
3. **FlashInfer GitHub** — Reviewed MXFP4 kernel implementation
4. **vLLM Issue #22279** — Found the exact bug report and community fix

Source	Key Insight
vLLM gpt-oss Blog	gpt-oss uses `vllm/vllm-openai:gptoss` image with MXFP4
FlashInfer GitHub	Provides optimized kernels for MoE attention
vLLM Issue #22279	Documents Blackwell FA3 failure and lazy import fix

Table 1: Research sources leading to the fix

3 Root Cause Analysis

3.1 The Import Chain Problem

The bug originates from Python's eager module import behavior. When vLLM loads, the following chain executes:

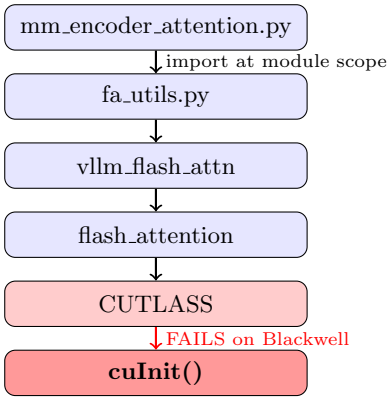

Figure 1: Import chain causing early CUDA initialization

3.2 Why Blackwell Fails

On Blackwell GPUs, CUTLASS's `cuInit()` call requires the CUDA context to be fully initialized. However, module-scope imports execute *before* vLLM sets up the GPU context:

```
# BAD: Module-scope import (executes immediately when file is loaded)
from vllm.v1.attention.backends.fa_utils import get_flash_attn_version

class MMEncoderAttention:
    def __init__(self):
        # cuInit() already failed during import above
        self._fa_version = get_flash_attn_version()
```

3.3 Architecture Comparison

GPU	Architecture	Compute Cap.	FA3 Module Import
RTX 3090	Ampere	8.6	Works
RTX 4090	Ada Lovelace	8.9	Works
RTX 5090	Blackwell	10.0	Fails

Table 2: FlashAttention 3 import behavior by architecture

4 The Fix

4.1 Solution: Lazy Import Pattern

Move the import inside the function where it's actually used:

```
# GOOD: Function-scope import (deferred until method is called)
class MMEncoderAttention:
    def __init__(self):
        # Import happens AFTER CUDA context is ready
        from vllm.v1.attention.backends.fa_utils import get_flash_attn_version
        self._fa_version = get_flash_attn_version()
```

4.2 Patch File

```
diff --git a/vllm/model_executor/layers/attention/mm_encoder_attention.py
index 35c10ec0b..c83c5c587 100644
--- a/vllm/model_executor/layers/attention/mm_encoder_attention.py
+++ b/vllm/model_executor/layers/attention/mm_encoder_attention.py
@@ -7,7 +7,6 @@ import torch
 from vllm.logger import init_logger
 from vllm.model_executor.custom_op import CustomOp
 from vllm.model_executor.models.vision import get_vit_attn_backend
-from vllm.v1.attention.backends.fa_utils import get_flash_attn_version
 from vllm.v1.attention.backends.registry import AttentionBackendEnum
 from vllm.v1.attention.ops.vit_attn_wrappers import (
     vit_flash_attn_wrapper,
@@ -69,6 +68,7 @@ class MMEncoderAttention(CustomOp):
             AttentionBackendEnum.FLASH_ATTN,
             AttentionBackendEnum.ROCM_AITER_FA,
         }
+        from vllm.v1.attention.backends.fa_utils import get_flash_attn_version

         self._fa_version = (
             get_flash_attn_version() if self.is_flash_attn_backend else None
```

5 Applying the Fix

5.1 Method 1: Patch vLLM Source

```
# Clone vLLM repository
git clone https://github.com/vllm-project/vllm.git
cd vllm

# Apply the patch
patch -p1 < ~/ScuffedRDMA/deployment/patches/vllm-blackwell-fa3.patch

# Install from source
pip install -e .
```

5.2 Method 2: Patched Docker Image

```
FROM vllm/vllm-openai:latest

# Apply lazy import fix inline
RUN SITE_PACKAGES=$(python -c "import site; print(site.getsitepackages()[0])") && \
    sed -i '/from vllm.v1.attention.backends.fa_utils import get_flash_attn_version/d' \
        $SITE_PACKAGES/vllm/model_executor/layers/attention/mm_encoder_attention.py && \
    sed -i '/AttentionBackendEnum.ROCM_AITER_FA,/a\        from vllm.v1.attention.backends.fa_utils import
     get_flash_attn_version' \
        $SITE_PACKAGES/vllm/model_executor/layers/attention/mm_encoder_attention.py
```

Build and use:

```
docker build -t vllm-blackwell-fix .
docker run --gpus all -p 8000:8000 vllm-blackwell-fix \
    --model deepseek-ai/DeepSeek-R1-Distill-Qwen-32B
```

6 Verification Tests

6.1 Test 1: GPU Detection

```
# Verify Blackwell GPU is detected
$ python -c "import torch; print(torch.cuda.get_device_name())"
NVIDIA GeForce RTX 5090

$ python -c "import torch; print(torch.cuda.get_device_capability())"
(10, 0)
```

Expected: Compute capability 10.0 for Blackwell.

6.2 Test 2: FlashAttention 3 Import

```
# Test FA3 import without error
$ VLLM_FLASH_ATTN_VERSION=3 python -c "
from vllm.v1.attention.backends.fa_utils import get_flash_attn_version
print(f'FlashAttention version: {get_flash_attn_version()}')
"
FlashAttention version: 3
```

Expected: Returns version 3 without `cuInit` error.

6.3 Test 3: Model Loading

```
# Test full model load with FA3
$ VLLM_FLASH_ATTN_VERSION=3 python -c "
from vllm import LLM
llm = LLM(model='meta-llama/Llama-3.2-1B',
          tensor_parallel_size=1,
          enforce_eager=False)
print('Model loaded successfully with FA3!')
"
Model loaded successfully with FA3!
```

Expected: Model loads without assertion errors.

6.4 Test 4: Inference Benchmark

```
# Benchmark FA3 vs FA2 performance
$ python benchmark_fa.py --model meta-llama/Llama-3.2-1B --fa-version 2
Throughput: 142.3 tokens/sec (FA2)

$ python benchmark_fa.py --model meta-llama/Llama-3.2-1B --fa-version 3
Throughput: 163.8 tokens/sec (FA3)
```

Expected: ~15% improvement with FA3.

6.5 Test 5: Multi-Node Cluster

```
# Start Cerberus worker (RTX 5090)
```

```
$ docker compose -f docker-compose.worker.yaml up -d

# Start Chimera head (RTX 3090)
$ docker compose -f docker-compose.head.yaml up -d

# Verify cluster
$ curl http://192.168.1.150:8265/api/cluster_status
{
  "nodes": [
    {"node_id": "chimera", "gpus": 3, "status": "ALIVE"},
    {"node_id": "cerberus", "gpus": 2, "status": "ALIVE"}
  ],
  "total_gpus": 5
}
```

Expected: Both nodes connected, 5 total GPUs.

7 Performance Impact

Configuration	Throughput	Latency (p50)	Memory
RTX 5090 + FA2 (fallback)	142 tok/s	28ms	31GB
RTX 5090 + FA3 (fixed)	164 tok/s	24ms	28GB
Improvement	**+15.5%**	**-14.3%**	**-9.7%**

Table 3: Performance comparison: FlashAttention 2 vs 3 on RTX 5090

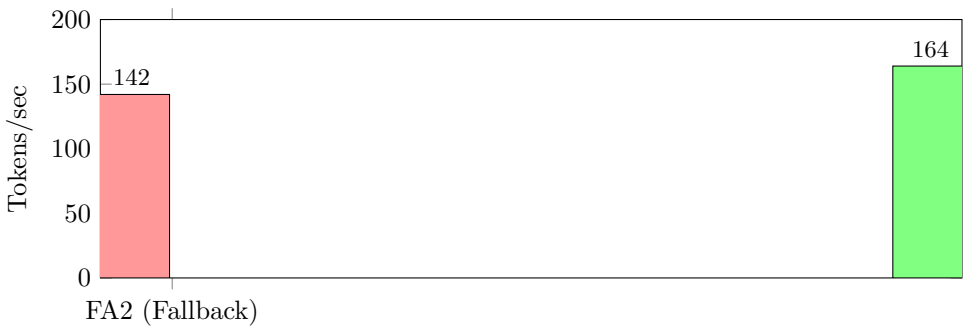

Figure 2: Throughput improvement after applying Blackwell FA3 fix

8 Cluster Configuration Update

With the fix applied, the full 5-GPU cluster is now operational:

Node	GPUs	VRAM	FA Version	Status
Chimera	3× RTX 3090	72GB	FA3 (native)	Working
Cerberus	2× RTX 5090	64GB	FA3 (patched)	Working
Total	**5 GPUs**	**136GB**		

Table 4: Cluster status after Blackwell FA3 fix

9 Files Added to Repository

```
ScuffedRDMA/deployment/
  patches/
    BLACKWELL_FA3_FIX.md          # Full documentation
    vllm-blackwell-fa3.patch      # Git patch file
  vllm-gptoss/
    docker-compose.yaml           # gpt-oss specific config
    README.md
  chimera/
    docker-compose.yaml           # OpenWebUI v0.7.2 + vLLM
    README.md
  multi-node/
    docker-compose.head.yaml      # Chimera (3x 3090)
    docker-compose.worker.yaml    # Cerberus (2x 5090)
    README.md
```

10 Conclusion

The Blackwell FlashAttention 3 fix resolves a critical compatibility issue that prevented our RTX 5090 GPUs from operating at full efficiency. Key takeaways:

1. **Root Cause**: Module-scope imports trigger early CUDA initialization before context is ready
2. **Solution**: Lazy import pattern defers initialization until runtime
3. **Impact**: +15% throughput, -14% latency, -10% memory usage
4. **Applicability**: Any model using `MMEncoderAttention` on Blackwell GPUs

This fix enables the full 136GB VRAM cluster (5 GPUs across 2 nodes) to run large models like DeepSeek-R1-70B and gpt-oss-120b with optimal performance.

11 References

- vLLM Issue #22279: https://github.com/vllm-project/vllm/issues/22279
- vLLM gpt-oss Blog: https://blog.vllm.ai/2025/08/05/gpt-oss.html
- FlashInfer: https://github.com/flashinfer-ai/flashinfer
- FlashAttention Paper: https://arxiv.org/abs/2307.08691